KYAMBOGO UNIVERSITY

FACULTY OF ENGINEERING · DEPARTMENT OF ELECTRICAL AND ELECTRONICS

TETE 3102: MICROCONTROLLERS AND APPLICATIONS

LABORATORY REPORT: TASK 7

Inter-Platform Link: STM32 Master to Arduino Uno Slave

Mukama Robert (26/U/ETE/21285/PE)
Nabudere Zerubabel (25/U/ETE/05465/PE)
Nanfuka Shakirah (25/U/ETE/05468/PE)
Kidega Wilkie Cephas (25/U/ETE/05460/PE)

Group Number: Group 4
Platforms: STM32F411RE & Arduino Uno
Course Supervisor: Dr. Mugerwa Dickson PhD

OBJECTIVE & LAB FOCUS

The objective of this laboratory task is to construct a heterogeneous, multi-voltage embedded communication link between a 32-bit ARM Cortex-M4 master (STM32F411RE at 3.3 V) and an 8-bit AVR slave (Arduino Uno at 5.0 V). Students implement bidirectional MOSFET level shifting, configure interrupt-driven slave event handlers (‘onReceive‘ and ‘onRequest‘), stream real-time analog potentiometer telemetry, render dynamic LCD bargraphs, and analyze clock stretching mechanics.

1 Hardware Interfacing & Level Shifting

Because the Arduino Uno operates at 5.0 V and the STM32 operates at 3.3 V, direct connection without isolation risks forward-biasing the internal protection diodes or injecting leakage current into 3.3 V rails. A bidirectional MOSFET level shifter circuit (N-channel BSS138) bridges the two domains.

Table 1: Heterogeneous Master-Slave Interconnection Topology

STM32 Master Pin	Level Shifter Pin	Arduino Slave Pin	Electrical Isolation / Function
+3.3V Rail	LV (Low Voltage Reference)	—	Powers the 3.3V side gate bias
—	HV (High Voltage Reference)	+5V Rail	Powers the 5V side gate bias
GND	GND	GND	Mandatory common electrical reference ground
PB8 (I2C1_SCL)	LV1 $\longleftrightarrow$ HV1	Pin A5 (SCL)	Bidirectional clock line with level conversion
PB9 (I2C1_SDA)	LV2 $\longleftrightarrow$ HV2	Pin A4 (SDA)	Bidirectional data line with level conversion
User Button (PC13)	—	Built-in LED (Pin 13)	Master user input toggles remote slave actuator
LCD1602 (PB8/PB9)	—	Potentiometer (A0)	Local LCD displays remote analog telemetry

2 Firmware Implementation

2.1 Arduino Uno Slave Firmware (Wire Event Driven)

The Arduino functions as an I^2C slave with 7-bit address `0x08`. It attaches hardware ISR callbacks: `receiveEvent` processes master command bytes, while `requestEvent` streams ADC readings.

Listing 1: Arduino Uno I2C Slave Implementation (Address 0x08)

```
#include <Wire.h>

const byte SLAVE_ADDR = 0x08;
const int POT_PIN = A0;
const int LED_PIN = 13; // Built-in LED

volatile byte led_state = 0;

void receiveEvent(int howMany) {
  while (Wire.available()) {
    byte cmd = Wire.read();
    if (cmd == 0x01) {
      led_state = 1;
      digitalWrite(LED_PIN, HIGH);
    } else if (cmd == 0x00) {
      led_state = 0;
      digitalWrite(LED_PIN, LOW);
    }
  }
}

void requestEvent() {
  // Read 10-bit ADC (0-1023) and scale to 8 bits (0-255)
  uint8_t pot_val = (uint8_t)(analogRead(POT_PIN) >> 2);
  Wire.write(pot_val); // Transmit telemetry to master
}

void setup() {
  pinMode(LED_PIN, OUTPUT);
  digitalWrite(LED_PIN, LOW);
  Wire.begin(SLAVE_ADDR);
  Wire.onReceive(receiveEvent);
  Wire.onRequest(requestEvent);
}

void loop() {
  // Keep loop empty; all I2C transactions handled via interrupts
}
```

2.2 STM32 Master Control Loop & Dynamic Bargraph Display

The STM32 acts as master, reading the onboard push-button (PC13) to toggle the Arduino's LED, polling the Arduino's potentiometer reading every 200 ms, and rendering an interactive visual bar on the LCD.

Listing 2: STM32 Master Firmware with Bargraph Rendering

```
#define ARDUINO_ADDR (0x08 << 1) // 7-bit 0x08 shifted to 0x10

void render_bargraph(uint8_t val) {
    char bar[17];
    uint8_t num_chars = (val * 16) / 255;
    for (uint8_t i = 0; i < 16; i++) {
        bar[i] = (i < num_chars) ? 0xFF : ' '; // 0xFF is solid black block on HD44780
    }
    bar[16] = '\0';
    lcd_set_cursor(1, 0);
    lcd_print(bar);
}

void master_telemetry_loop(void) {
    uint8_t tx_cmd = 0;
    uint8_t rx_pot = 0;
    char text[17];
```

```
18
19      while (1) {
20          /* 1. Check user button (active LOW) to toggle remote Arduino LED */
21          if (HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_13) == GPIO_PIN_RESET) {
22              tx_cmd ^= 0x01; // Toggle state
23              HAL_I2C_Master_Transmit(&hi2c1, ARDUINO_ADDR, &tx_cmd, 1, 50);
24              HAL_Delay(250); // Debounce delay
25          }
26
27          /* 2. Read 8-bit potentiometer telemetry from Arduino slave */
28          if (HAL_I2C_Master_Receive(&hi2c1, ARDUINO_ADDR, &rx_pot, 1, 50) == HAL_OK) {
29              snprintf(text, sizeof(text), "POT VAL: %3d/255", rx_pot);
30              lcd_set_cursor(0, 0);
31              lcd_print(text);
32              render_bargraph(rx_pot);
33          } else {
34              lcd_set_cursor(0, 0); lcd_print("SLAVE DISCONNECT");
35              lcd_set_cursor(1, 0); lcd_print("STATUS: TIMEOUT ");
36          }
37
38          HAL_Delay(200);
39      }
40  }
```

3 Bus Frame Diagrams

1. Master Write Frame (Toggle Command)

S	Slave Address + W (0x10)	ACK	Command Byte (0x01 / 0x00)	ACK	P
Master	Master ($0001\ 0000_2$)	Slave	Master	Slave	Master

2. Master Read Frame (Potentiometer Telemetry)

S	Slave Address + R (0x11)	ACK	Data Byte (e.g. 0x80)	NACK	P
Master	Master ($0001\ 0001_2$)	Slave	Slave	Master	Master

4 Mandatory Task Questions & Detailed Solutions

Task Question 1

Which side stretches the clock, if any, and why?

Answer: The **Arduino Uno slave** stretches the clock. When the STM32 master completes transmitting the address byte (`0x11`), the AVR hardware asserts an interrupt request. The ATmega328P hardware holds the SCL line LOW while invoking the interrupt service routine (‘requestEvent‘). Because SCL is open-drain, the master cannot drive SCL HIGH until the AVR ISR finishes writing the data byte into the ‘TWDR‘ (Two-Wire Data Register) and clears the ‘TWINT‘ flag, which releases SCL.

Task Question 2

What happens if the Arduino slave is physically powered off? What exact HAL status and error flags are returned to the STM32?

Answer: When the Arduino slave is unpowered:

- **Physical Wire State:** Address byte `0x10` or `0x11` is pulsed onto the bus, but no device pulls SDA low on the 9th clock. SDA floats HIGH via the pull-up resistor (NACK).

- **Returned Function Status:** `HAL_I2C_Master_Transmit` and `HAL_I2C_Master_Receive` return **`HAL_ERROR`** (`0x01`).
- **Underlying Hardware Error Flag:** Inspecting the handle via `HAL_I2C_GetError(&hi2c1)` returns **`HAL_I2C_ERROR_AF`** (`0x04`, Acknowledge Failure).

5 Conclusion

Task 7 proves the robustness of the I^2C multi-voltage architecture. With appropriate MOSFET level shifting and interrupt-driven firmware, heterogeneous microcontrollers exchange commands and telemetry reliably without bus conflicts or clock desynchronization.